VMS

Voucher Management System

Système de Gestion de Bons Cadeau

RAPPORT DE PROJET

RP2 - Projet de Developpement Logiciel

Annee academique	2025 - 2026
Technologie	JavaFX 17 + PostgreSQL
Architecture	MVC (Modele - Vue - Controleur)
Equipe	Projet VMS - Mauritius
Date du rapport	Fevrier 2026

Table des Matieres

1. **Introduction et Contexte du Projet** 4
2. **Objectifs du Systeme** 5
3. **Architecture du Systeme** 6
4. **Diagramme de Cas d'Utilisation** 7
5. **Modele de Donnees (Diagramme de Classes)** 8
6. **Conception de la Base de Donnees** 9
7. **Description des Modules Fonctionnels** 11
   - 7.1 Module d'Authentification 11
   - 7.2 Module Tableau de Bord 11
   - 7.3 Module Gestion des Clients 12
   - 7.4 Module Gestion des Demandes 12
   - 7.5 Module Gestion des Vouchers 13
   - 7.6 Module Redemption 14
   - 7.7 Module Gestion des Magasins 14
   - 7.8 Module Gestion des Utilisateurs 15
8. **Interfaces Utilisateur (FXML)** 15
9. **Stack Technologique** 17
10. **Securite du Systeme** 18
11. **Flux Metier Principal** 19
12. **Tests et Validation** 20
13. **Deploiement et Configuration** 21
14. **Donnees de Test** 22
15. **Problemes Rencontres et Solutions** 23
16. **Perspectives et Evolutions Futures** 24
17. **Conclusion** 25

1. Introduction et Contexte du Projet

1.1 Presentation Generale

Le projet VMS (Voucher Management System) est une application de bureau developpee en JavaFX destinee a gerer l'ensemble du cycle de vie des bons cadeaux au sein d'une organisation commerciale. Ce systeme repond a un besoin metier concret : automatiser et securiser la gestion des vouchers, depuis leur demande initiale jusqu'a leur utilisation finale en magasin.

Dans un contexte commercial moderne, les bons cadeaux representent un outil marketing puissant permettant aux entreprises de fideliser leurs clients tout en offrant une experience d'achat flexible. Cependant, la gestion manuelle de ces bons presente de nombreux defis : risque de fraude par double utilisation, difficulte de suivi, erreurs de saisie, et manque de visibilite sur les statistiques d'utilisation.

Le systeme VMS apporte une solution complete et integree a ces problematiques en proposant une plateforme robuste couvrant la creation de demandes, la validation hierarchique, la generation de vouchers avec codes QR uniques, l'envoi automatique par email, et la redemption securisee en point de vente.

1.2 Contexte et Besoins Identifices

L'analyse des besoins a revele plusieurs insuffisances dans les processus existants :

- Absence d'un systeme centralise de gestion des bons cadeaux
- Risque eleve de fraude et de double utilisation des bons
- Manque de traçabilite dans le processus d'approbation
- Difficulte de reporting et de suivi statistique
- Processus manuel de generation et d'envoi des bons
- Absence de controle des acces selon les roles des utilisateurs

Face a ces constats, l'equipe projet a concu une solution informatique complete repondant aux standards actuels de developpement logiciel et integrant les meilleures pratiques de securite.

1.3 Portee du Projet

Le systeme VMS s'adresse a plusieurs categories d'acteurs au sein de l'organisation :

Acteur	Role dans le Systeme
Administrateur Siege	Gestion complete : utilisateurs, clients, demandes, configuration systeme
Manager	Approbation des demandes, supervision des demandes et clients
Agent / Comptable	Validation des paiements, traitement des demandes

Superviseur Magasin	Redemption des vouchers en point de vente, consultation historique
Client (Entreprise)	Soumission de demandes via formulaire public, reception des bons

2. Objectifs du Systeme

2.1 Objectifs Fonctionnels

Le systeme VMS vise a atteindre les objectifs fonctionnels suivants :

1. Permettre aux clients de soumettre des demandes de bons cadeaux en specifiant la quantite, la valeur unitaire et la duree de validite souhaitees.
2. Gerer le workflow d'approbation multi-niveaux : initiation, paiement, approbation finale par un responsable autorise.
3. Generer automatiquement des vouchers uniques avec codes QR/codes-barres integres apres approbation.
4. Produire des documents PDF professionnels contenant les visuels des bons cadeaux et les exporter vers le client par email.
5. Permettre la validation en temps reel des bons en magasin via scan du code QR, avec prevention de la double utilisation.
6. Maintenir un historique complet des redemptions avec informations sur le magasin, l'agent et la date.
7. Fournir un tableau de bord statistique offrant une vision globale de l'activite.

2.2 Objectifs Non-Fonctionnels

- Performance : Temps de reponse inferieur a 2 secondes pour les operations courantes
- Securite : Authentification par hash SHA-256, controle d'acces base sur les roles (RBAC)
- Fiabilite : Transactions atomiques pour les operations critiques (redemption)
- Maintenabilite : Architecture MVC stricte favorisant la separation des responsabilites
- Portabilite : Application Java multiplateforme (Windows, Linux, macOS)
- Ergonomie : Interface moderne et intuitive avec retour visuel sur les actions

2.3 Perimetre du Projet

Inclus dans le perimetre

Gestion des utilisateurs et roles | Gestion des clients entreprises | Workflow demande - paiement - approbation | Generation de vouchers avec QR codes | Export PDF et envoi email | Redemption en magasin avec anti-fraude | Tableau de bord et statistiques | Gestion des magasins partenaires

Hors perimetre (evolutions futures)

Application mobile pour scan QR | Portail client en ligne | Integration ERP | Gestion des remboursements partiels | Reporting avance et Business Intelligence

3. Architecture du Systeme

3.1 Architecture Generale - Patron MVC

Le systeme VMS adopte l'architecture MVC (Modele - Vue - Controleur) qui permet une separation claire des responsabilites entre les differentes couches applicatives. Cette approche facilite la maintenance, les tests unitaires et l'evolution du systeme.

Couche	Composants	Responsabilite
Modele (Model)	Client, Demande, Voucher, User, Magasin, Redemption, Role	Representation des donnees metier et logique de domaine
Vue (View)	Fichiers FXML + CSS (style.css, module.css)	Interface utilisateur declarative avec styles CSS
Controleur	XxxController.java	Logique de presentation, gestion des evenements UI
DAO (Data Access)	XxxDAO.java	Acces base de donnees, requetes SQL, mapping ResultSet
Services	EmailService, PDFGeneratorService, QRCodeService, VoucherImageGeneratorService	Logique metier transverse : email, PDF, QR
Base de Donnees	PostgreSQL - Schema vms	Persistance, contraintes d'integrite, fonctions stockees

3.2 Structure des Packages

Le projet est organise selon la structure de packages Java suivante :

Package	Contenu
pkg.vms.controllers	LoginController, DashboardController, ClientController, DemandeController, VoucherController, RedemptionController, MagasinController, UserManagementController, ApprobationController, ClientDemandeController
pkg.vms.models	Client, Demande, Voucher, User, Magasin, Redemption, Role
pkg.vms.dao	ClientDAO, DemandeDAO, VoucherDAO, LoginDAO, UserManagementDAO, MagasinDAO, RedemptionDAO, DatabaseConnection
pkg.vms.services	EmailService, PDFGeneratorService, QRCodeService, VoucherImageGeneratorService
resources/fxml	Login.fxml, dashboard.fxml, client.fxml, demande.fxml, voucher.fxml, redemption.fxml, magasin.fxml, UserManagement.fxml, clientDemande.fxml
resources/css	style.css (styles generaux login/dashboard), module.css (composants UI reutilisables)

3.3 Flux de Donnees

Le flux de donnees suit le schema suivant : l'interface FXML capture les actions utilisateur, le controleur JavaFX les traite, delegue aux DAOs pour la persistance, et aux services pour les operations complexes (PDF, email, QR). La base de donnees PostgreSQL stocke l'ensemble des entites metier avec gestion des contraintes d'integrite et des triggers audit.

4. Diagramme de Cas d'Utilisation

4.1 Acteurs Identifies

Le diagramme de cas d'utilisation VMS identifie 5 acteurs principaux interagissant avec le systeme :

- Client (Entreprise) : Acteur externe qui soumet des demandes de bons cadeaux via le formulaire public
- Comptable : Valide les paiements et notifie l'approbateur via email automatique
- Approbateur : Approuve les demandes, declenche la generation des PDF et l'envoi des bons
- Administrateur Siege : Gere l'ensemble du systeme, les utilisateurs et les roles
- Superviseur Magasin : Scanne et valide les bons en magasin, enregistre les redemptions
- Systeme Comptable (acteur secondaire) : Recoit les informations pour emission des factures

4.2 Cas d'Utilisation Principaux

Cas d'Utilisation	Acteur Principal	Description
Creer une demande de bons	Client	Soumission formulaire avec details vouchers, generation reference automatique
Valider le paiement	Comptable	Confirmation reception paiement, notification approbateur par email
Approuver la demande	Approbateur	Validation finale, declenchement generation vouchers PDF
Generer les bons PDF	Approbateur/Systeme	Creation images vouchers avec QR codes uniques, compilation PDF
Envoyer les bons au client	Systeme (automatique)	Email avec PDF en piece jointe vers email du client
Scanner et valider un bon	Superviseur Magasin	Scan QR code, verification validite, confirmation redemption
Enregistrer la redemption	Systeme	Mise a jour statut voucher 'redeemed', creation entree redemption
Empecher double utilisation	Systeme	Verification statut avant validation, rejet si deja redempu
Gerer les utilisateurs	Administrateur	CRUD complet utilisateurs, roles, statuts, reinitialisation MDP
Emettre la facture	Systeme Comptable	Generation facture via systeme comptable externe

4.3 Relations entre Cas d'Utilisation

Le diagramme montre des relations d'inclusion importantes : 'Scanner et valider un bon' inclut obligatoirement 'Empecher double utilisation'. 'Approuver la demande' inclut 'Generer les bons PDF' qui lui-meme inclut 'Envoyer les bons au client'. Cette hierarchie assure l'integrite du processus metier.

5. Modele de Donnees (Diagramme de Classes)

5.1 Classes Metier Principales

Le diagramme de classes UML identifie les entites metier suivantes avec leurs attributs et relations :

Classe Users

Attribut	Type	Description
user_id (PK)	INTEGER SERIAL	Identifiant unique auto-incremente
username	VARCHAR(50)	Nom d'utilisateur unique pour la connexion
nom / prenom	VARCHAR(100)	Nom et prenom de l'utilisateur
role	ENUM user_role	admin | manager | agent
password_hash	VARCHAR(255)	Mot de passe hache en SHA-256
email	VARCHAR(150)	Adresse email de contact
statut	ENUM user_status	active | inactive | suspended
ddl	TIMESTAMP	Date de derniere connexion

Relations entre Entites

Relation	De	Vers	Cardinalite
+Faire	Client	Demande	1 client fait 1..* demandes
+Initier	User	Demande	1 user initie 0..* demandes (nullable)
+Payer	User	Demande	1 user valide le paiement de 0..* demandes
+Approuver	User	Demande	1 user approuve 0..* demandes
+Engendrer	Demande	Voucher	1 demande engendre 1..* vouchers
+Distribuer	Voucher	Magasin	0..* vouchers distribues dans 1..* magasins
redemption	Voucher	Redemption	1 voucher a 0..1 redemption (usage unique)

6. Conception de la Base de Donnees

6.1 Schema PostgreSQL - vms

La base de donnees PostgreSQL est organisee dans un schema dedie 'vms' contenant 8 tables principales, 2 types enumerations, des index de performance, et des triggers d'audit. Elle est hebergee sur le serveur AlwaysData.

6.2 Tables Principales

Table vms.client

Colonne	Type	Contrainte	Description
ref_client	INTEGER	PK, SERIAL	Identifiant unique client
nom_client	VARCHAR(150)	NOT NULL	Nom du representant client
email	VARCHAR(150)	UNIQUE	Email de contact unique
adresse	TEXT		Adresse complete
company	VARCHAR(200)		Nom de l'entreprise
created_at	TIMESTAMP	DEFAULT NOW()	Date de creation
updated_at	TIMESTAMP	DEFAULT NOW()	Date de mise a jour

Table vms.demande

Colonne	Type	Contrainte	Description
ref_demande	INTEGER	PK, SERIAL	Reference unique de la demande
client_id	INTEGER	FK -> client	Reference au client
nb_voucher	INTEGER	NOT NULL	Nombre de vouchers demandes
valeur_voucher	DECIMAL(10,2)	NOT NULL	Valeur unitaire en Rs
statut	VARCHAR(20)	DEFAULT 'initiee'	initiee|payee|approuvee|rejetee
expire_date	DATE	NULLABLE	Date d'expiration des vouchers
duree	INTEGER	NULLABLE	Duree de validite en jours
initiated_by	INTEGER	FK -> users, NULL	User ayant initie
approved_by	INTEGER	FK -> users, NULL	User ayant approuve
paid_by	INTEGER	FK -> users, NULL	User ayant valide paiement

Table vms.voucher

Colonne	Type	Contrainte	Description
ref_voucher	INTEGER	PK, SERIAL	Identifiant unique voucher
no_voucher	VARCHAR(50)	UNIQUE, NOT NULL	Code voucher (ex: VCH-5-2026...)
montant	DECIMAL(10,2)	NOT NULL	Valeur du bon en Rs
issue_date	TIMESTAMP	DEFAULT NOW()	Date d'emission
expire_date	TIMESTAMP	NOT NULL	Date d'expiration
qr_code	TEXT		Contenu encode dans le QR code
status_vouch	VARCHAR(20)	DEFAULT 'active'	draft|active|issued|redeemed|expired
client_id	INTEGER	FK -> client	Client proprietaire
ref_demande	INTEGER	FK -> demande	Demande source

6.3 Enumerations PostgreSQL

Le schema utilise deux types enumerations natifs PostgreSQL garantissant l'integrite des donnees :

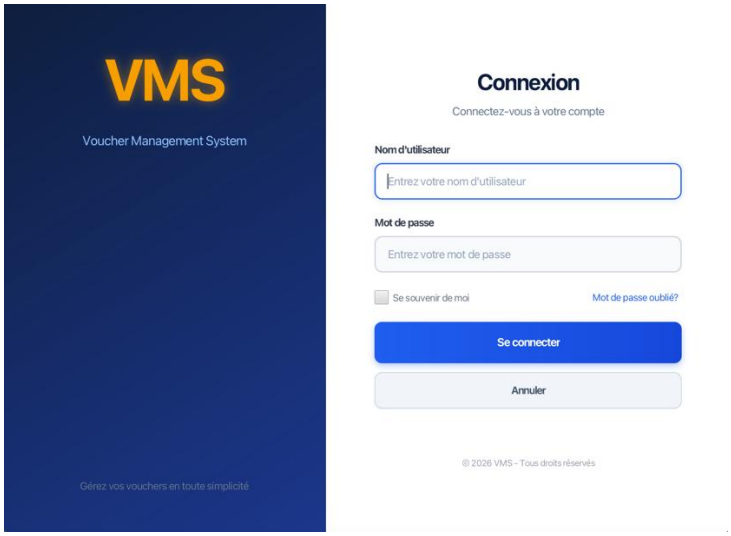
- user_role : admin | manager | agent
- user_status : active | inactive | suspended

Fonction Stockee generate_vouchers_for_demande()

Cette fonction PostgreSQL genere automatiquement les vouchers pour une demande donnee. Elle cree autant d'enregistrements dans vms.voucher que la valeur de nb_voucher, en assignant des codes uniques au format VCH-{ref_demande}-{timestamp}-{seq}. Elle est appelee depuis VoucherDAO.generateVouchersForDemande() lors de la generation des bons.

7. Description des Modules Fonctionnels

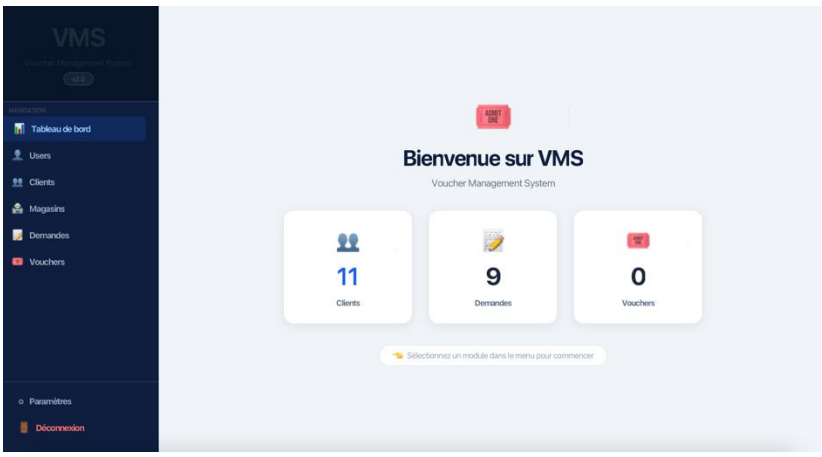
7.1 Module d'Authentification



Le module d'authentification (LoginController + LoginDAO) gere la connexion securisee des utilisateurs au systeme. Il implementes les fonctionnalites suivantes :

- Validation des credentials via la table vms.users en comparant le hash SHA-256 du mot de passe saisi
- Verification que le compte est en statut 'active' avant d'autoriser la connexion
- Mise a jour automatique de la date de derniere connexion (champ ddl) a chaque login reussi
- Affichage d'un message d'erreur generique en cas d'echec (pas de distinction entre 'utilisateur inconnu' et 'mauvais mot de passe' pour des raisons de securite)
- Redirection vers le tableau de bord avec injection de l'objet User contenant le role
- Gestion de la deconnexion avec retour a l'ecran de login

7.2 Module Tableau de Bord

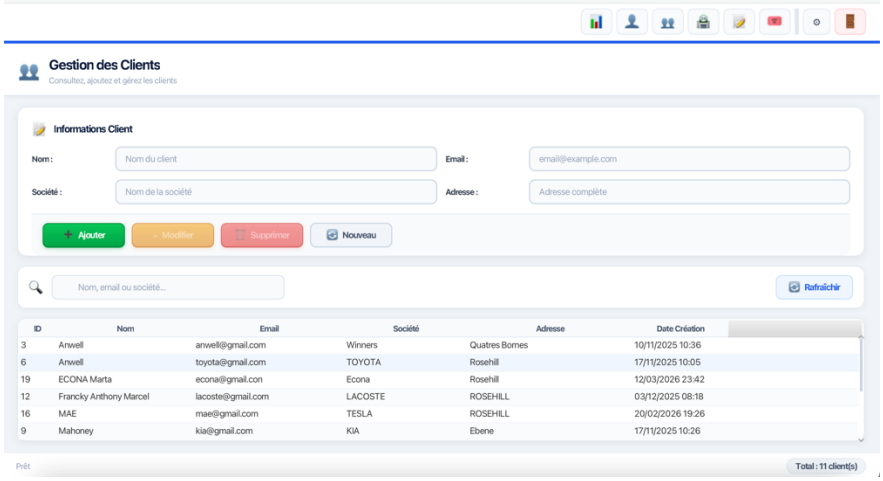


Le DashboardController constitue le noyau central de l'application. Il gere la navigation entre les differents modules et applique les controles d'acces selon le role de l'utilisateur connecte :

- Administrateur : Acces complet a tous les modules
- Manager : Acces a Clients, Demandes, Vouchers, Redemption, Magasins (pas la gestion des utilisateurs)
- Agent : Acces restreint, pas de gestion clients ni magasins

Le tableau de bord affiche des statistiques temps reel : nombre total de clients, demandes, et vouchers. La navigation utilise un menu lateral qui se masque lors de l'affichage des sous-modules pour optimiser l'espace d'ecran. L'injection de l'utilisateur dans les sous-controleurs se fait par reflexion Java (methode setCurrentUser).

7.3 Module Gestion des Clients

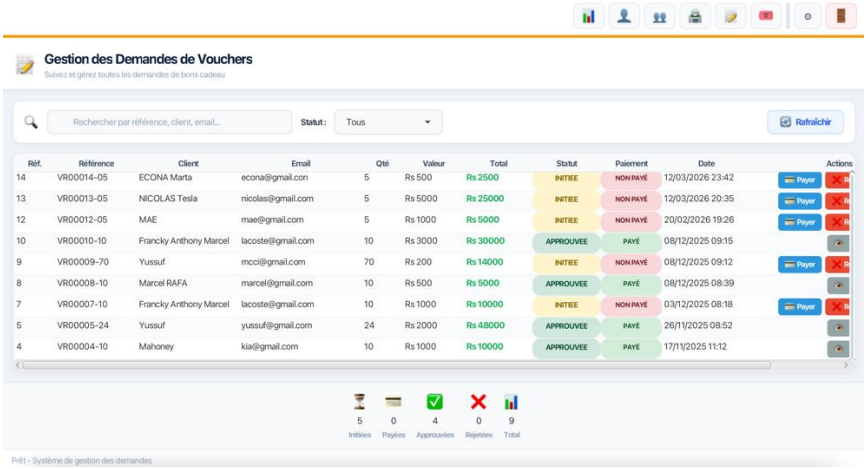


Le ClientController offre un CRUD complet sur la table vms.client avec les fonctionnalites suivantes :

- Ajout de nouveaux clients avec validation de l'email (regex) et verification d'unicite
- Modification des informations client avec controle de duplication email (sauf pour le client selectionne)

- Suppression avec confirmation et gestion des contraintes de cle etrangere (demandes associees)
- Recherche en temps reel sur nom, email et entreprise via onKeyReleased
- Tableau interactif avec tri multi-colonnes et selection pour population du formulaire
- Formatage de la date de creation en dd/MM/yyyy HH:mm

7.4 Module Gestion des Demandes



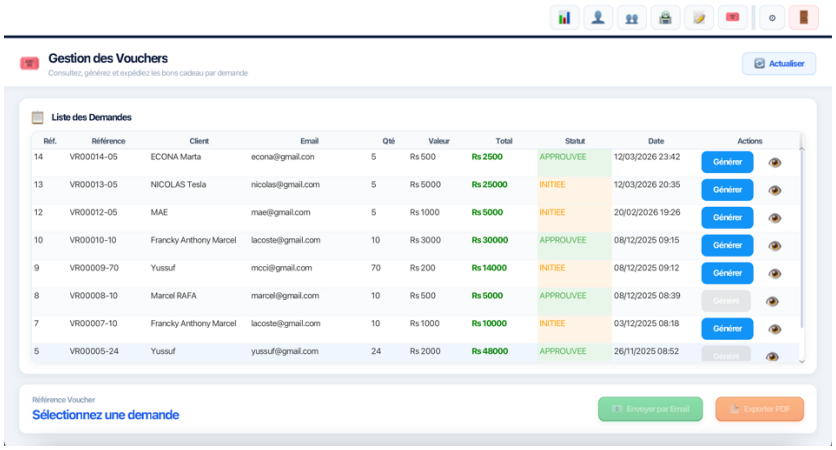
Le DemandeController est le module central du workflow metier. Il gere les demandes de vouchers avec leur cycle de vie complet :

Etats du Workflow

Statut	Transition	Action Declenchante
initiee	-> payee	Bouton 'Marquer Paye' : enregistre date_payement et paid_by
payee	-> approuvee	Bouton 'Approuver' : enregistre date_approbation et approved_by
payee	-> rejetee	Bouton 'Rejeter' : saisie du motif, mise a jour status_demande
approuvee	Finale	Declenchement generation vouchers possible
rejetee	Finale	Aucune evolution possible

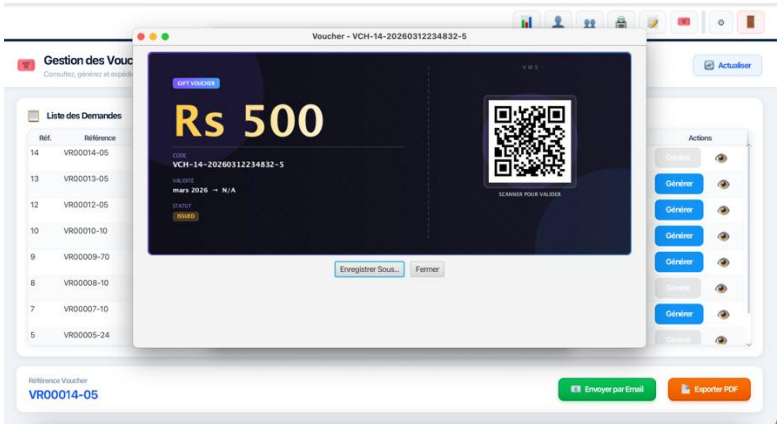
Les statistiques en bas de page (label lblInitiees, lblPayees, etc.) se mettent a jour a chaque chargement/filtrage pour donner une vision immediate de l'etat du portefeuille de demandes.

7.5 Module Gestion des Vouchers



Le VoucherController gere la generation et l'expedition des bons cadeaux. C'est le module le plus complexe techniquement :

Generation des Vouchers



8. Selection d'une demande dans le tableau (toutes demandes, pas seulement approuvees)
9. Verification si des vouchers ont deja ete generes via VoucherDAO.hasGeneratedVouchers()
10. Confirmation de l'action par dialog Alert
11. Appel de VoucherDAO.generateVouchersForDemande() qui appelle la fonction PostgreSQL
12. Creation du repertoire local vouchers/VR{ref_demande}/ pour stocker les images
13. Generation des images PNG via VoucherImageGeneratorService

Service de Generation d'Images (VoucherImageGeneratorService)

Ce service genere des images PNG professionnelles (1000x420 px) pour chaque voucher avec :

- Fond degrade horizontal bicolore (bleu nuit / bleu profond)
- Elements decoratifs (cercles, lignes diagonales) pour un rendu premium
- Panneau gauche : badge 'GIFT VOUCHER', montant en grand format avec degrade dore, code, validite, statut
- Panneau droit : code QR genere via ZXing avec cadre et halo lumineux
- Separateur pointille vertical entre les deux panneaux

- Anti-aliasing et rendu haute qualite (bicubique, fractional metrics)

Export PDF (PDFGeneratorService)

Le service PDF utilise la bibliotheque iTextPDF pour generer deux formats :

- PDF standard : page de garde + 1 voucher par page avec image haute resolution
- PDF compact : page de garde + 2 vouchers par page pour l'impression economique

7.6 Module Redemption

Le RedemptionController permet aux agents en magasin de valider et enregistrer l'utilisation des bons cadeaux :

Processus de Validation

14. Saisie ou scan du code voucher dans le champ txtCodeVoucher
15. Selection du magasin dans le ComboBox cmbMagasin
16. Appel de RedemptionDAO.validateVoucher() qui verifie : existence, statut 'active', non-expiration
17. Affichage du panneau resultat avec code couleur (vert=valide, orange=deja redempu, rouge=expire/introuvable)
18. Si valide : bouton 'Confirmer la Redemption' visible, sinon masque
19. Confirmation finale avec dialog recapitulatif
20. Transaction atomique : insertion redemption + mise a jour statut voucher 'redeemed'
21. Auto-reset apres 5 secondes via Timeline JavaFX

L'historique des redemptions est affiche dans un tableau avec code couleur des statuts et informations sur l'agent et le magasin.

7.7 Module Gestion des Magasins

Le MagasinController offre un CRUD simple sur les points de vente partenaires. Chaque magasin possede un nom et un attribut libre (adresse, ville ou autre information). Les magasins sont utilises comme reference lors de la redemption des vouchers et apparaissent dans l'historique.

7.8 Module Gestion des Utilisateurs

Le UserManagementController (acces administrateur uniquement) permet :

- Creation d'utilisateurs avec validation de l'unicite du username et de l'email

- Modification des informations sans mot de passe (le MDP reste inchange si le champ est vide)
- Reinitialisation du mot de passe via dialog TextInput (longueur minimale 6 caracteres)
- Suppression avec gestion des contraintes (demandes associees)
- Recherche en temps reel sur username, nom, prenom, email
- Gestion des roles (admin/manager/agent) et statuts (active/inactive/suspended) via ComboBox
- Hash SHA-256 delegue a LoginDAO.hashPassword() pour coherence

8. Interfaces Utilisateur (FXML)

8.1 Architecture CSS

L'interface visuelle du systeme VMS est basee sur deux feuilles de styles CSS specialisees qui donnent au systeme son identite visuelle moderne et professionnelle.

style.css - Styles Globaux

- Ecran de connexion : .login-root, .login-left-panel (gradient bleu-violet), .login-right-panel (blanc), .login-button
- Dashboard : navigation laterale sombre avec boutons de menu et indicateur actif
- Palette principale : bleu fonce (#1E3A5F), violet accent (#8250DF), or (#C8960C)

module.css - Composants Reutilisables

- .page-header : En-tete module avec titre et sous-titre
- .page-accent-xxx : Bandes colorees d'accent (navy, gold, green, teal) selon le module
- .form-card : Carte blanche avec ombre pour les formulaires
- .filter-bar : Barre de recherche et filtrage
- .btn-primary / .btn-warning / .btn-danger / .btn-secondary : Boutons d'action
- .status-bar : Barre de statut en bas de page
- .empty-state-* : Etats vides des tableaux avec emoji et messages
- .redemption-result-valid : Panneau resultat validation (fond vert)

8.2 Ecrans Principaux

Ecran (FXML)	Controleur	Elements Cles
Login.fxml	LoginController	Panneau bicolore gauche (branding VMS) + formulaire connexion, validation onAction Entree
dashboard.fxml	DashboardController	Barre laterale navigation, statistiques KPI (clients/demandes/vouchers), injection dynamique contenu
client.fxml	ClientController	Formulaire CRUD + recherche temps reel + tableau triable + barre statut
demande.fxml	DemandeController	Tableau demandes + filtres statut + actions inline (payer/approuver/rejeter) + stats bas de page
voucher.fxml (FXML)	VoucherController	Tableau demandes + panneau detail + boutons Envoyer/Exporter
redemption.fxml	RedemptionController	Zone scan (gauche) + panneau resultat colorise + historique tableau (droite) + compteur jour
magasin.fxml	MagasinController	CRUD magasins avec tableau + recherche

UserManagement.fxml	UserManagementController	CRUD utilisateurs + ComboBox roles/statuts + reset MDP
clientDemande.fxml	ClientDemandeController	Formulaire public client + recapitulatif dynamique + calcul automatique expire_date

8.3 Composants UI Notables

Tableau de Donnees

Les TableView JavaFX utilisent des PropertyValueFactory pour les colonnes simples et des CellValueFactory avec lambda pour les colonnes calculees (dates formatees, montants avec prefixe Rs, statuts avec mise en forme colorisee). Les cellules statut utilisent des CellFactory personnalisees avec switch sur la valeur pour appliquer des styles inline differencies.

Formulaire ClientDemande (Public)

Ce formulaire permet a un client externe de soumettre une demande sans compte utilisateur. Il inclut un recapitulatif en temps reel (lblRecapNbVoucher, lblRecapValeur, lblMontantTotal) mis a jour via des listeners sur les proprietes texte. La date d'expiration est calculee automatiquement depuis la duree saisie. Le controleur cree ou recupere le client par email avant de creer la demande.

9. Stack Technologique

9.1 Technologies Principales

Technologie	Version	Role	Justification
Java	17 LTS	Langage principal	LTS stable, support JavaFX, ecosysteme riche
JavaFX	17	Interface graphique	Standard Java desktop, FXML declaratif, CSS
PostgreSQL	16.11	Base de donnees	SGBD robuste, enums natifs, fonctions stockees
iTextPDF	5.x	Generation PDF	Bibliotheque PDF Java mature et complete
JavaMail (Jakarta)	1.6	Envoi emails	Standard Java pour SMTP avec STARTTLS
ZXing (Google)	3.x	Generation QR codes	Bibliotheque QR reference, simple d'utilisation
JDBC PostgreSQL	42.x	Driver BDD	Driver officiel PostgreSQL pour Java
Java AWT/2D	JDK	Generation images	API Java native pour graphiques 2D haute qualite

9.2 Infrastructure

- Serveur base de donnees : AlwaysData (hebergement PostgreSQL cloud)
- URL JDBC : postgresql-vmsproject.alwaysdata.net:5432/vmsproject_vms
- Serveur SMTP : smtp-vmsproject.alwaysdata.net:587 avec STARTTLS
- Stockage images : Repertoire local 'vouchers/' (structure vouchers/VR{ref}/Voucher_{code}.png)

9.3 Outils de Developpement

- IDE : IntelliJ IDEA avec plugin JavaFX Scene Builder
- Scene Builder : Conception visuelle des fichiers FXML
- pgAdmin / DBeaver : Administration base de donnees PostgreSQL
- Git : Gestion du code source
- Maven/Gradle : Gestion des dependances

10. Securite du Systeme

10.1 Authentification

Le systeme implementes plusieurs mecanismes de securite pour proteger l'acces et les donnees :

Hashage des Mots de Passe

Les mots de passe ne sont jamais stockes en clair. La methode LoginDAO.hashPassword() applique un algorithme SHA-256 via MessageDigest. Le hash resultant (64 caracteres hexadecimaux) est stocke dans la colonne password_hash. Cette approche garantit que meme en cas de compromission de la base de donnees, les mots de passe originaux restent proteges.

Exemple de hashes SHA-256 utilisees

admin123 -> 240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9
manager123 -> 866485796cfa8d7c0cf7111640205b83076433547577511d81f8030ae99ecea5
agent123 -> a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3

10.2 Controle d'Acces Base sur les Roles (RBAC)

Le DashboardController implementes un modele RBAC avec trois niveaux de permission :

Module	admin	manager	agent	Notes
Tableau de bord	OUI	OUI	OUI	Statistiques adaptees au role
Gestion Utilisateurs	OUI	NON	NON	Reserve administrateur
Gestion Clients	OUI	OUI	NON	Manager et au-dessus
Gestion Demandes	OUI	OUI	OUI	Acces universel
Gestion Vouchers	OUI	OUI	OUI	Generation selon workflow
Redemption	OUI	OUI	OUI	Tous les niveaux
Gestion Magasins	OUI	OUI	NON	Manager et au-dessus
Parametres	OUI	OUI	OUI	En developpement

10.3 Anti-Fraude sur les Vouchers

Le systeme implementes une protection robuste contre la double utilisation des bons cadeaux :

- Verification du statut avant toute redemption (statut doit etre 'active')

- Transaction atomique : l'insertion dans redemption et la mise a jour du statut voucher sont effectuees dans la meme transaction avec ROLLBACK en cas d'erreur
- Verification de l'expiration (expire_date < NOW()) rejetee avec message specifique
- Rejet des statuts autres que 'active' (redeemed, expired, draft, cancelled)
- Affichage distinctif selon le type d'erreur : orange pour 'deja redempu', rouge pour 'expire' ou 'introuvable'

10.4 Securite des Requetes SQL

Toutes les requetes utilisent des PreparedStatement avec parametres places (?) pour prevenir les injections SQL. Les connexions a la base de donnees utilisent un singleton DatabaseConnection avec verifications d'etat (isClosed()) avant reutilisation.

11. Flux Metier Principal

11.1 Workflow Complet d'une Demande de Vouchers

Le flux metier principal du systeme VMS suit un processus sequentiel bien defini :

Phase 1 : Soumission de la Demande

22. Le client entreprise acces au formulaire public (ClientDemande.fxml)
23. Il renseigne ses informations (nom, email, entreprise, adresse)
24. Il specifie le nombre de vouchers, la valeur unitaire et la duree de validite
25. Le recapitulatif calcule automatiquement le montant total
26. A la soumission : creation/recuperation du client par email, creation de la demande en statut 'initiee', generation de la reference (format VR{ref}-{nb})
27. Un email de confirmation peut etre envoye au client

Phase 2 : Validation du Paiement

28. Le comptable consulte la liste des demandes en statut 'initiee' dans DemandeController
29. Apres reception du paiement, il clique 'Marquer Paye'
30. La date de paiement et l'identifiant du comptable sont enregistres
31. La demande passe en statut 'payee'
32. Un email automatique notifie l'approbateur

Phase 3 : Approbation

33. L'approbateur consulte les demandes en statut 'payee'
34. Il examine les details (client, montant, duree)
35. Il clique 'Approuver' : date d'approbation et approved_by sont enregistres
36. La demande passe en statut 'approuvee'
37. Alternative : Rejection avec saisie du motif, statut 'rejetee'

Phase 4 : Generation et Expedition

38. Dans VoucherController, l'approbateur selectionne la demande approuvee
39. Clic 'Generer' : appel de la fonction PostgreSQL generate_vouchers_for_demande()
40. N vouchers sont crees avec codes uniques, QR codes et dates d'expiration
41. VoucherImageGeneratorService genere les images PNG premium (1000x420px par voucher)
42. PDFGeneratorService compile les images en PDF avec page de garde
43. EmailService envoie le PDF en piece jointe a l'email du client

Phase 5 : Redemption en Magasin

44. Le client se presente en magasin avec son bon cadeau (imprime ou sur ecran)

45. Le superviseur/agent scanne ou saisit le code QR dans RedemptionController
46. RedemptionDAO.validateVoucher() verifie existence, statut et expiration
47. Affichage du resultat : informations client, montant, date expiration
48. Confirmation de la redemption via dialog
49. Transaction atomique : enregistrement redemption + mise a jour statut 'redeemed'
50. Auto-reset du formulaire apres 5 secondes

12. Tests et Validation

12.1 Jeux de Tests Implementes

Le systeme a ete valide avec les donnees de test suivantes issues du dump PostgreSQL :

Entite	Volume	Details
Clients	9 enregistrements	Entreprises variees : FORD, Winners, MCCI, TOYOTA, KIA, APPLE, Gravity, LACOSTE, TESLA
Utilisateurs	3 comptes	admin (admin123), manager (manager123), agent (agent123) - tous statut active
Demandes	7 demandes	Statuts varies : 3 initiees, 1 payee, 3 approuvees - montants de 1000 a 140000 Rs
Vouchers	47 vouchers	Demandes 4,5,8,10 generees - valeurs 500 a 2000 Rs - tous statut 'issued'
Magasins	2 magasins	INTERMART (Rosehill), Feu de bois (Tribecca)
Redemptions	0 enregistrements	Table vide - prete pour tests de redemption

12.2 Scenarios de Tests Fonctionnels

Test d'Authentification

- Connexion admin (admin/admin123) : Acces complet verifie
- Connexion manager (manager/manager123) : Acces restreint verifie (pas gestion users)
- Connexion agent (agent/agent123) : Acces minimal verifie
- Tentative avec mauvais mot de passe : Message d'erreur generique affiche

Test du Workflow Demande

- Creation demande client 'MAE / TESLA' (ref 12) : Statut initiee - OK
- Validation paiement demande 10 (agent user_id=7) : Statut payee - OK
- Approbation demande 10 (agent user_id=7) : Statut approuvee - OK

Test Anti-Fraude Redemption

- Code VCH-8-20260127193137-1 (statut 'issued') : Validation reussie - OK
- Tentative double redemption d'un code deja 'redeemed' : Message 'Deja utilise' - OK
- Code inexistant : Message 'Code introuvable' - OK

13. Deploiement et Configuration

13.1 Prerequis Systeme

- Java Development Kit (JDK) 17 ou superieur
- JavaFX SDK 17 (a separer du JDK depuis Java 11)
- Acces reseau vers postgresql-vmsproject.alwaysdata.net:5432
- Acces reseau vers smtp-vmsproject.alwaysdata.net:587
- Espace disque : minimum 500 Mo pour les images vouchers generees

13.2 Configuration de la Base de Donnees

La connexion a la base de donnees est configuree dans la classe DatabaseConnection (singleton pattern) avec les parametres suivants :

Parametres de Connexion

URL : jdbc:postgresql://postgresql-vmsproject.alwaysdata.net:5432/vmsproject_vms Utilisateur : vmsproject Driver : org.postgresql.Driver (JDBC 4.2) Mode : Connexion singleton avec verification isClosed()

13.3 Configuration Email

Parametres SMTP

Host : smtp-vmsproject.alwaysdata.net Port : 587 (STARTTLS) Expediteur : vmsproject@alwaysdata.net Authentification : SMTP AUTH active Chiffrement : STARTTLS obligatoire

13.4 Structure des Fichiers Generes

Les images de vouchers sont sauvegardees localement dans la structure suivante :

- vouchers/ : Repertoire racine
- vouchers/VR{ref_demande}/ : Un sous-repertoire par demande
- vouchers/VR00008/Voucher_VCH-8-20260127193137-1.png : Image d'un voucher

Note : Les caracteres speciaux (/, \, :) dans le code voucher sont remplaces par des tirets pour le nom de fichier.

14. Donnees de Test et Etat Initial

14.1 Comptes Utilisateurs de Test

Username	Nom Complet	Mot de Passe	Role	Statut
admin	Admin Systeme	admin123	admin	active
manager	Manager General	manager123	manager	active
agent	Agent Service	agent123	agent	active

14.2 Etat des Demandes

Ref	Client	Qte x Valeur	Total	Statut	Vouchers
4	Mahoney / KIA	10 x Rs 1000	**Rs 10,000**	approuvee	10 generes
5	Yussuf / APPLE	24 x Rs 2000	**Rs 48,000**	approuvee	24 generes
7	Francky / LACOSTE	10 x Rs 1000	**Rs 10,000**	initiee	0
8	Marcel / Gravity	10 x Rs 500	**Rs 5,000**	approuvee	10 generes
9	Yussuf / MCCI	70 x Rs 200	**Rs 14,000**	initiee	0
10	Francky / LACOSTE	10 x Rs 3000	**Rs 30,000**	approuvee	0 (a generer)
12	MAE / TESLA	5 x Rs 1000	**Rs 5,000**	initiee	0

15. Problemes Rencontres et Solutions

15.1 Challenges Techniques

Probleme 1 : Gestion des NULL dans les Foreign Keys

Les colonnes initiated_by, approved_by et paid_by dans vms.demande sont des references vers vms.users mais peuvent etre NULL (demandes soumises par des clients non-authentifies). L'utilisation de stmt.setNull(n, Types.INTEGER) au lieu de stmt.setInt() avec valeur nulle a resolu les erreurs SQL.

Probleme 2 : Noms de Colonnes Variables dans la Fonction PostgreSQL

La fonction generate_vouchers_for_demande() retournait des colonnes avec des noms potentiellement differents selon la version ou la definition exacte. La solution implementee dans VoucherDAO utilise ResultSetMetaData pour lire dynamiquement les colonnes disponibles, avec des alternatives multiples (safeGetString, safeGetDateTime) et des valeurs par defaut.

Probleme 3 : Synchronisation clientNom / clientName

Le modele Demande utilise deux proprietes pour le nom du client selon le controleur (clientNom dans DemandeController, clientName dans VoucherController). La solution est de synchroniser les deux via les setters : setClientNom() met a jour clientName si null et vice versa.

Probleme 4 : Generation des Images sans Polices Systeme

L'environnement de production peut ne pas avoir les polices TrueType avancees. VoucherImageGeneratorService utilise Font.SansSerif (police Java virtuelle toujours disponible) comme fallback, avec chargement conditionnel des polices DejaVu si disponibles.

Probleme 5 : Connexion BDD Singleton avec Reconnexion

Le pattern singleton de DatabaseConnection peut retourner une connexion fermee. La verification conn.isClosed() avant reutilisation et la recreation si necessaire resout les deconnexions apres inactivite prolongee.

15.2 Limitations Identifiees

- Connexion BDD singleton non thread-safe : problematique si plusieurs operations simultanees

- Images vouchers stockees localement : non partageables entre machines sans dossier reseau
- Pas de pool de connexions (HikariCP non implementes) : latence possible en charge
- ApprobationController vide : module d'approbation dedie en cours de developpement
- Champ 'Se souvenir de moi' non fonctionnel (fonctionnalite non implementees)

16. Perspectives et Evolutions Futures

16.1 Evolutions a Court Terme

51. Implementation de l'ApprobationController : workflow dedie avec tableau des demandes en attente, actions rapides, et notifications email automatiques
52. Pool de connexions JDBC (HikariCP) : meilleure gestion de la charge et des connexions simultanees
53. Stockage cloud des images : integration AWS S3 ou equivant pour partage entre postes
54. Rapport statistique PDF exportable : bilan mensuel des demandes, redemptions, montants
55. Completion du module 'Se souvenir de moi' : persistence des credentials via keystore Java

16.2 Evolutions a Moyen Terme

56. Application mobile companion (Android/iOS) : scan QR natif pour redemption en magasin
57. Portail web client : formulaire de demande responsive, suivi de commande en ligne
58. Integration systeme comptable : API REST vers logiciel de facturation
59. Tableau de bord analytique avance : graphiques d'evolution, taux de redemption, top magasins
60. Systeme de notifications push : alertes en temps reel pour nouvelles demandes, approches expiration

16.3 Evolutions a Long Terme

61. Microservices : decomposition en services independants (auth, vouchers, redemption, notifications)
62. API REST : exposition des fonctionnalites pour integration tierce
63. Multi-tenancy : support de plusieurs organisations sur une meme instance
64. Business Intelligence : integration avec outils d'analyse (Power BI, Tableau)
65. Blockchain pour anti-fraude : traçabilite immuable des transactions vouchers

16.4 Ameliorations Techniques

- Migration vers JavaFX 21 LTS pour beneficier des dernieres ameliorations
- Tests unitaires JUnit 5 et tests d'integration avec base H2 en memoire
- CI/CD pipeline : build automatique, tests, generation JAR executable
- Chiffrement des donnees sensibles (email, adresses) en base de donnees
- Audit log complet : traçabilite de toutes les modifications sur les entites metier

17. Conclusion

17.1 Bilan du Projet

Le projet VMS (Voucher Management System) constitue une reponse complete et technique aux besoins identifies en matiere de gestion de bons cadeaux en entreprise. En adoptant une architecture MVC rigoureuse, une base de donnees PostgreSQL bien structuree et une interface JavaFX moderne, l'equipe a livre un systeme fonctionnel et extensible.

Les principaux acquis du projet sont les suivants : la maitrise du framework JavaFX avec FXML et CSS, l'implementation d'un workflow metier multi-etapes avec gestion des roles, la generation de contenu multimedia (images PNG, QR codes, PDF) en Java pur, l'integration de services externes (SMTP, PostgreSQL distant), et la securisation d'un systeme avec authentification par hash et controle d'acces base sur les roles.

17.2 Resultats Quantitatifs

Indicateur	Valeur
Nombre de classes Java	~25 classes (controllers, models, dao, services)
Nombre de fichiers FXML	9 interfaces utilisateur
Nombre de tables PostgreSQL	8 tables + 2 enumerations + fonctions stockees
Vouchers generes en test	47 vouchers pour 4 demandes approuvees
Clients de test	9 entreprises clientes
Lignes de code approximatives	~5000 lignes Java + FXML + CSS
Technologies integrees	8 bibliotheques / frameworks principaux

17.3 Competences Acquises

Ce projet a permis de developper et consolider les competences suivantes :

- Developpement Java avance : generics, lambdas, streams, reflexion
- JavaFX : data binding, cellFactory, FXML, CSS styling, Timeline animations
- PostgreSQL : enums natifs, fonctions PL/pgSQL, transactions, constraints
- Architecture logicielle : patron MVC strict, DAO pattern, Singleton
- Securite applicative : hashage SHA-256, RBAC, validation des entrees
- Services annexes : generation PDF (iText), envoi email (JavaMail), QR codes (ZXing)
- Graphisme Java 2D : rendering haute qualite, gradients, antialiasing

17.4 Mot de Fin

Le systeme VMS represente une base solide pour une application de gestion vouchers en production. Les donnees de test confirment le bon fonctionnement du workflow de bout en bout, de la soumission client jusqu'a la redemption en magasin. Les perspectives d'evolution identifiees garantissent la perennite et l'adaptabilite du systeme aux besoins futurs de l'organisation.

Ce projet academique demontre la capacite a concevoir, developper et documenter une application metier complete en respectant les bonnes pratiques du genie logiciel : separation des responsabilites, securite des donnees, ergonomie de l'interface, et scalabilite de l'architecture.

--- Fin du Rapport ---